

IE 492 — Senior Design Project · Final Report

Stop the Fire!

*A network optimization approach to wildfire
containment in the Köyceğiz–Marmaris region*

Team

Efe Ergen · 2021402168
Emre Barutçu · 2023402219
Mehmet Efe Aloğlu · 2020402024
Kerem Külünkoğlu · 2023402222

Supervisor

Prof. Tınaz Ekim

Department of Industrial Engineering
Boğaziçi University
May 2026

Contents

Abstract	3
Özet	3
1. Introduction	4
1.1 The industrial engineering problem	4
1.2 Identification, analysis, and solution methodology in brief	4
1.3 Expected improvements	5
1.4 Brief summary of conclusions	5
1.5 Contents of the report	5
2. Problem Definition, Requirements and Limitations	6
2.1 What seems to be the problem	6
2.2 What was done to understand the causes	6
2.3 Needs and requirements of the system / customer	7
2.4 Limitations and constraints	7
2.5 Data gathered and used in the identification phase	8
2.6 Context diagram	8
2.7 Performance criteria and potential improvements	9
3. Analysis for Solution / Design Methodology	10
3.1 Literature overview	10
3.2 Alternative solution / design approaches	11
3.3 Assumptions	11
3.4 Brief overview of the selected approach	12
3.5 IE skills, tools, techniques and methods integrated	12
4. Development of Alternative Solutions	13
4.1 Common framework	13
4.2 Strategy 1 — Max Degree (local, baseline)	14
4.3 Strategy 2 — Max White Neighbours (local, refined)	14
4.4 Strategy 3 — Min Cut Edge Front (structural, baseline)	14
4.5 Strategy 4 — Min Damage Cut (structural, reformulated)	15
4.6 Strategy 5 — Betweenness Front (structural, low-cost)	16
4.7 Strategy 6 — Hybrid Density Aware (rule-based switching)	16
4.8 Strategy 7 — One-Step Lookahead (rollout)	16
4.9 Strategy 8 — Random (sanity baseline)	17
4.10 Strategy portfolio at a glance	17
5. Comparison of Alternatives and Recommendation	18
5.1 Numerical study	18
5.2 Proposed solution and justification	22
5.3 Further assessment of the recommended solution	23
6. Suggestions for a Successful Implementation	24
6.1 How the design can be implemented	24
6.2 Integration with the overall system	28

6.3 Revision cadence	29
7. Conclusions and Discussion	29
7.1 IE tools, techniques and methods used	29
7.2 Merits and significance of the design	30
7.3 Economic, environmental, ethical and societal impacts	30
References	31
Appendix A — Detailed Test Configuration	32
Appendix B — Strategy Implementation Pointers	32
Appendix C — Web Suite Endpoints	33

3. Analysis for Solution / Design Methodology

3.1 Literature overview

The starting point of the literature is Hartnell’s 1995 introduction of the Firefighter Problem at the 25th Manitoba Conference on Combinatorics [1]. A fire breaks out at a vertex of a graph; at each step a firefighter protects up to k vertices, then the fire spreads to all unprotected neighbours of burning vertices. The goal is to minimise the number of burned vertices. Finbow and MacGillivray [2] catalogue the early structural results: on infinite grids the problem becomes a tractable shape-design question; on trees with maximum degree three it is already NP-hard. Cai, Cheng, Verbin and Zhou [3] give a tight 2-approximation for trees and an $e/(e - 1)$ -approximation for the general problem under certain assumptions. On planar graphs of maximum degree at most four, the problem is APX-hard [4], which closes the door on a polynomial-time exact algorithm for our setting.

Approximation and heuristic work then splits into two families. The *local greedy* family (max-degree, max-white-neighbours, save-the-most-savable) makes a single-step decision from a local feature of the fire front. The *structural* family treats the spread as a flow problem and computes min vertex/edge cuts between the burning set and a target set of “valuable” vertices. Anshelevich, Chakrabarty, Hate and Swamy [5] study cut-based protection strategies and prove that, for any instance, an offline optimal protection plan can be encoded as an integer program with a polynomial number of cut constraints; in practice the IP is intractable for our graph sizes within field time-budgets, which is why heuristics dominate.

A third family is *lookahead and rollout*. Bertsekas’s rollout framework [7] shows that simulating a base policy for one or two steps inside the decision can capture multi-step dynamics that pure local heuristics miss; in the firefighter literature this is closely related to the subexponential exact algorithms of Cai, Verbin and Yang [6], which exploit the same idea that bounded-depth lookahead on the spread tree carries most of the information needed for a good protection plan. The practical cost is one or two extra simulation calls per candidate, which is acceptable when the candidate set is pruned to a few dozen.

On the spatial side, the continuous fire-spread modelling literature (rate-of-spread models and operational descendants such as FARSITE) is largely orthogonal to our work: those models predict the physical front of a continuous fire under wind and topography, whereas we treat propagation as a discrete graph process. The bridge between the two paradigms is the vertex-placement step. We use Lloyd’s k-means algorithm [8] with density-weighted centroids to place graph vertices on dense forest patches extracted from the ESA WorldCover satellite raster [9], and we drop candidate edges that cross physical barriers (water, bare strips, settlements) derived from OpenStreetMap [10]. The result is a discrete network that respects the geometry of the real landscape without committing to a continuous fire-physics model. The climate-driven motivation for studying Mediterranean wildfire containment specifically is established by Turco et al. [12], who link summer Mediterranean fire activity to drought intensity, and is updated annually by the EFFIS report for Europe, the Middle East and North Africa [11].

3.2 Alternative solution / design approaches

We considered four families of approach before settling on the heuristic-ensemble route.

(A) Exact integer programming. Formulate the problem as a mixed-integer program over a state-space of (vertex, time) and solve with Gurobi or CPLEX. Rejected: even for $n = 30$ the IP has tens of thousands of binary variables, and proven NP-hardness implies that the gap to optimality cannot be closed for the graph sizes we care about ($n \approx 100$ for a 600 km^2 region).

(B) Reinforcement learning. Train a policy network with Q-learning or PPO that takes the (state, k) tuple as input and emits an action. Rejected for two reasons: (i) it requires a training distribution that may not match the operational distribution; (ii) the recommendation has to be *explainable* to a forestry planner, and a black-box policy is harder to defend than a heuristic with a one-line rationale.

(C) Pure local greedy. Implement only the max-degree and max-white-neighbours heuristics and accept that the strategy might be suboptimal on long-thin geometries. Rejected because the empirical gap between local greedy and lookahead is large (3–7 percentage points on synthetic instances, 4 points on Köyceğiz).

(D) Heuristic ensemble + recommendation. Implement multiple heuristics, run them all in parallel on the user’s instance, and let a topology-aware recommendation engine pick a winner with a short justification. Selected: it gives the planner comparable evidence, lets us avoid betting the whole project on a single heuristic, and aligns with the 14 April directive that “different graphs need different strategies.”

3.3 Assumptions

The model rests on four explicit assumptions, all carried unchanged from the problem definition and confirmed in the meeting notes.

1. **Discrete and uniform propagation.** Time progresses in steps; at each step a burning vertex ignites *all* unprotected neighbours simultaneously. This is the standard Firefighter rule. Real fires spread asymmetrically (wind, slope), and the discrete-uniform assumption inflates the apparent spread speed in some directions and dampens it in others. We argue that the asymmetries average out for *strategy ranking*: a strategy that is better than another under uniform spread tends to remain better under modest perturbations.
2. **Irreversibility of damage.** Burnt vertices cannot recover. This matches the long restoration times of Mediterranean forests (canopy reconstitution takes 15–50 years).
3. **Absolute immunity of protected vertices.** Once green, a vertex is immune. Field reality is grey: a vertex defended by a single team can still burn if the surrounding load is too high. Our model treats “protected” as a planning decision that succeeds, abstracting away the operational fail-rate.
4. **Full observability.** The decision algorithm knows the full graph topology and the exact state at every time step. In the field the planner has line-of-sight reports plus aerial surveillance; the assumption is approximately right for the planning horizon of one to two hours.

3.4 Brief overview of the selected approach

The selected approach is a four-stage pipeline: **bbox** → **forest graph** → **strategy ensemble** → **recommended defence schedule** (Figure 2).

- **Stage 1 (map-to-graph)**. A user-supplied bounding box is intersected with the ESA WorldCover raster to produce a forest mask. The mask is the set of 10 m cells whose neighbourhood contains at least 55% forest fraction. Vertex count is parameterised by an area-per-vertex slider (default 500 ha/vertex, giving 101 vertices in Köyceğiz). Lloyd’s k-means with k-means++ initialisation and 14 iterations places centroids on the densest cells; centroids that snap into non-forest holes are pushed to the nearest dense cell.
- **Stage 2 (edge filter)**. A Delaunay triangulation of the vertex set produces candidate edges. Five physical rules then filter the candidates: bare-strip > 60 m, settlement buffer < 600 m, river buffer < 80 m, water-on-path, low vegetation density. For Köyceğiz, 278 candidate edges are pruned to 172 active and 106 blocked. Blocked edges are kept in the data structure (greyed out in the UI) so the planner can see the natural barriers.
- **Stage 3 (strategy ensemble)**. Seven strategies plus a random baseline run on the resulting graph for a user-chosen fire origin and budget k . The strategies are described in detail in §4.
- **Stage 4 (recommendation)**. A topology fingerprint (graph diameter, average degree, density, articulation count, k value) classifies the instance into `delanay_like`, `long_thin`, `obstacles`, or `hex_like`. A score table then ranks the strategies for that class; the winner and the runner-up are surfaced together with a one-sentence reason.

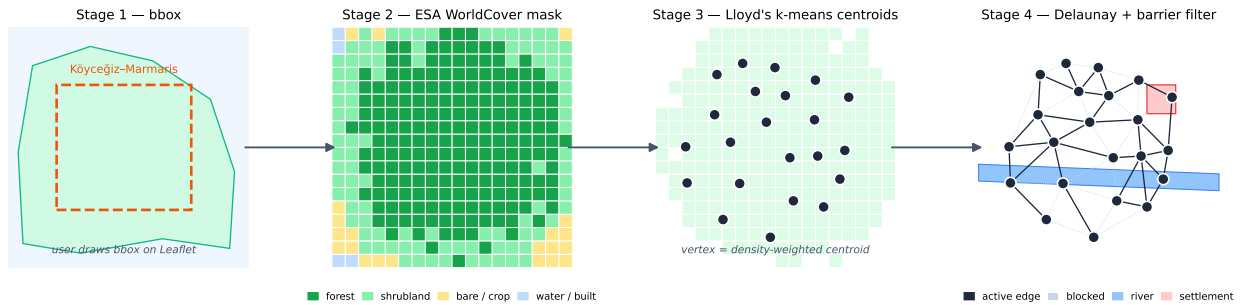


Figure 2: The four-stage map-to-graph pipeline: bounding box → ESA WorldCover forest mask → Lloyd’s k-means vertex placement → Delaunay edges filtered by five physical barrier rules (rivers, settlements, bare strips). Active edges are black, blocked edges light grey.

The output is consumed by the web suite (§6.1), which also animates the chosen strategy turn by turn.

3.5 IE skills, tools, techniques and methods integrated

- **Network optimisation:** flow-based min-cut, vertex cuts, BFS shells, augmented source/sink construction.

- **Combinatorial heuristics:** greedy, dynamic priority ranking, rollout-style lookahead, hybrid switching rules.
 - **Computational geometry:** Delaunay triangulation (`scipy.spatial.Delaunay`), Voronoi-equivalent dual graphs, Lloyd’s algorithm for density-weighted centroids.
 - **GIS / remote sensing:** raster ingestion with `rasterio`, projection between WGS84 and UTM, ESA WorldCover land-cover class semantics, Overpass query construction for OpenStreetMap.
 - **Simulation:** discrete-event, protect-then-spread engine; deterministic per-turn snapshot wrapper for animation.
 - **Statistical experiment design:** 3 sizes $\times$ 18 seeds $\times$ 11 starting points $\times$ 8 strategies = 4,752 runs, paired by graph and starting point to control for instance variance.
 - **Software engineering:** FastAPI backend with disk-based tile and Overpass caching, Vite/React/TypeScript frontend with Leaflet map widget, Dockerised build for deployment.
 - **Decision-support design:** topology fingerprinting, score-based recommendation, explainability via a one-line rationale.
-

4. Development of Alternative Solutions

This section walks through the seven informed strategies (plus the random baseline) implemented in `backend/firefighter_engine.py`. Each strategy is described using the *find* $\rightarrow$ *check* $\rightarrow$ *decide* template that our supervisor asked for on 31 March: what does the strategy *find*, how does it *check* the candidates, how does it *decide* on the top k ? The eight strategies share the same input contract: at the start of each turn the strategy receives the graph G , the state map (white/red/green), and the budget k ; it returns a priority-ranked list of white vertices, and the engine protects the first k that are still white before applying the spread step.

4.1 Common framework

Every strategy starts from the same three pieces of information:

1. **State map.** Per-vertex labels: white (safe), red (burning), green (protected).
2. **Fire front F .** All white vertices that have at least one red neighbour. Only vertices in F (or in their neighbourhood) are eligible to be protected.
3. **Budget k .** The number of vertices the engine will protect this turn ($k = 2$ in the baseline).

A turn proceeds as: `strategy.select` $\rightarrow$ engine protects top- k still-white candidates $\rightarrow$ fire spreads to every white neighbour of every red vertex $\rightarrow$ loop until F is empty (the fire is contained). Figure 3 illustrates this protect-then-spread cycle on a 13-vertex toy graph.

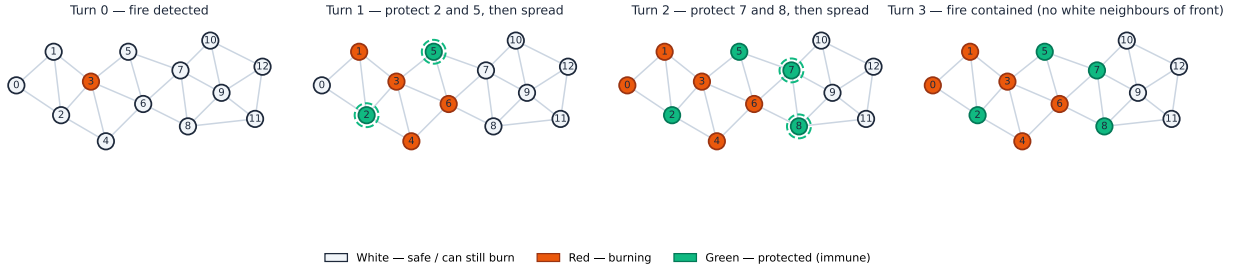


Figure 3: Protect-then-spread mechanics on a 13-vertex toy graph with $k = 2$ per turn. The strategy protects two white vertices (dashed green rings) each turn; the engine then spreads fire to every white neighbour of every red vertex. The campaign ends at turn 3 when no red vertex has a white neighbour.

4.2 Strategy 1 — Max Degree (local, baseline)

Philosophy. “Highly connected vertices are important; if I let one burn, the fire branches in many directions.”

- **Find.** All white vertices in the fire front F .
- **Check.** Score = total degree (red, white and green neighbours all counted).
- **Decide.** Sort F by degree descending; pick the first k .

This is the obvious first-thought heuristic. It captures the intuition that hubs are dangerous to lose. Its weakness is that “degree” is a static topological measure that does not see the *colour* of the neighbours. A vertex with eight neighbours, seven of which are already burnt, scores high under Max Degree even though protecting it saves almost nothing.

4.3 Strategy 2 — Max White Neighbours (local, refined)

Philosophy. “If I protect this vertex, how many white vertices does that actually save next turn?”

- **Find.** All white vertices in F .
- **Check.** Score = number of *white* neighbours. Ties broken by total degree.
- **Decide.** Sort F by score descending; pick the first k .

This is the patch to Max Degree’s blind spot. It asks the effective question (how much fresh forest will this protection actually shield) rather than the topological one (how many edges does this vertex have). Empirically it beats Max Degree on every topology we tested, by 1–3 percentage points on synthetic instances and by 10 percentage points on the real Köyceğiz map. It became the local-greedy benchmark for the rest of the project.

4.4 Strategy 3 — Min Cut Edge Front (structural, baseline)

Philosophy. “Where is the narrowest bridge between the burning region and the safe region? Protect the vertices that sit on it.”

- **Find.** Build an augmented graph: a super-source connected to every red vertex, a super-sink connected to every white vertex outside F . The candidate cut is the minimum edge cut between source and sink.
- **Check.** For each cut edge, count the white endpoints that lie in F . A vertex’s score is the number of cut edges incident on it.
- **Decide.** Sort F by score descending; pick the first k .

This strategy correctly identifies *where the bottleneck is* but suffers a structural mismatch: min-cut is an *edge*-based answer, but we protect *vertices*. Saving half the endpoints of a cut leaves the other half active, and the fire flows through. Empirically Min Cut Edge Front is one of the weakest informed strategies (43.4% burned on synthetic average), only marginally better than random.

4.5 Strategy 4 — Min Damage Cut (structural, reformulated)

Philosophy. “Stop minimising cut cost. Maximise the ratio of vertices saved per unit of cut cost, across multiple candidate cuts.”

This strategy came directly out of the 14 April supervisor feedback. The original Min Cut Vertex Front (no longer in the deployed ensemble, since Min Damage Cut now supersedes it) was answering the wrong question: it computed a single vertex cut to a single distant target and protected that. Min Damage Cut instead enumerates a *family* of candidate cuts and chooses the most effective one.

- **Find.** Compute BFS distances from the red set to every white vertex. For each shell depth $d \in \{2, 3, 4\}$, define the target set $T_d = \{v : \text{dist}(\text{red}, v) \geq d\}$. For each T_d , build the augmented graph (super-source on red, super-sink on T_d) and compute the minimum *vertex* cut C_d .
- **Check.** $\text{Score}(C_d) = |\text{saved}(C_d)| / |C_d|$, where $\text{saved}(C_d)$ is the set of white vertices that become disconnected from red once C_d is protected. This is the *saved-per-cost ratio*; the supervisor’s phrasing was “how many regions does this cut save per defended vertex?”
- **Decide.** Pick the C_d with the highest ratio. Within that cut, order the vertices so that those in F come first, then by score.

On the development benchmark used during the reformulation, the retired Min Cut Vertex Front baseline burned 39.4% of vertices; Min Damage Cut brought that down to 30.3%, a 23% relative improvement. That is the headline number from the April 14 $\rightarrow$ April 21 cycle. In the final deployed ensemble (Table 5.1, where the obsolete Vertex Front baseline is no longer present), Min Damage Cut sits at 42.6% on average against 43.4% for Min Cut Edge Front, a smaller within-family lead because the comparator is itself a different cut formulation. The dramatic individual results are still there: one long-thin instance shows a single protected vertex saving twelve others, a $12\times$ ratio. The reason the strategy does not dominate on average is that the median cut size is around six vertices, but we only have $k = 2$ firefighters per turn, so the fire often eats half of the cut before we can finish placing it.

4.6 Strategy 5 — Betweenness Front (structural, low-cost)

Philosophy. “Fire is a kind of traffic. Find the vertices that the most shortest paths pass through, in the still-living subgraph.”

- **Find.** Build the *white subgraph* (delete every red and green vertex from G). Compute betweenness centrality of every vertex in this subgraph.
- **Check.** Score = betweenness value. Fire-front vertices with high betweenness are the ones the fire is most likely to use as a corridor to reach distant parts of the network.
- **Decide.** Sort F by betweenness descending; pick the first k .

Betweenness is a global structural measure that has the same intuition as min-cut (find the bottleneck) but is much cheaper to compute on small graphs (`networkx.betweenness_centrality` runs in $O(VE)$). Empirically it placed second overall on synthetic average (38.5% burned) at one-quarter of the cost of Min Damage Cut. It is our recommended structural strategy for budget-constrained runs.

4.7 Strategy 6 — Hybrid Density Aware (rule-based switching)

Our first attempt at the if-else hybrid that the supervisor asked for on 24 March takes a single scalar signal — front-density, defined as $|F| / |\text{white}|$ — and routes the turn to one of two sub-strategies. When front-density falls below 0.18 (a narrow front threading through a still-mostly-white graph), the turn is handed to Min Damage Cut, on the intuition that a small cut can finish the job. Otherwise (a broad front consuming a large fraction of the remaining graph), the turn goes to Max White Neighbours, which behaves better in saturated regimes. The hybrid then returns whichever sub-strategy’s priority ranking applies.

Empirically Hybrid Density Aware ranks fourth on the synthetic average (42.2% burned). The threshold of 0.18 was set by inspection rather than tuning and is the obvious lever to adjust. A more principled version would switch on a direct estimate of cut quality — for instance, use the cut whenever $\text{cut_size} \leq k$ and $\text{saved/cost} \geq 3$, otherwise fall back to greedy — but we did not test that variant within the project timeline. We flag it in §5 as a near-term improvement.

4.8 Strategy 7 — One-Step Lookahead (rollout)

Philosophy. “Ask not what is good now, but what is good two turns from now. Simulate the engine inside the decision.”

This is the strategy that wins both synthetic and real-map benchmarks.

- **Find.** Take the top six candidates from F by Max-White-Neighbours score. From these six, enumerate all $C(6, 2) = 15$ unordered pairs (v_1, v_2) .
- **Check.** For each pair (v_1, v_2) , copy the current state. Mark v_1 and v_2 as green. Run one spread step. From the resulting front, pick the two best vertices under Max-White-Neighbours and protect them too. Run one more spread step. Count burned.

- **Decide.** Pick the pair with the lowest burned count after two turns. Return that pair first, then the remaining four candidates in score order.

The strategy works for compounding reasons. Most fundamentally, it matches the engine’s real dynamic: the engine alternates protect $\rightarrow$ spread, and Lookahead alternates the same way inside its decision, whereas the greedy and cut strategies operate on a snapshot. On top of that it is *pair-aware* — it sees the interaction between the two protected vertices in the same turn, which a single-vertex score function cannot — and the second spread step lets it pick up indirect effects (protecting v_1 stops the fire from reaching v_1 ’s neighbours’ neighbours next turn, which a one-step view cannot see).

The cost is modest: top-k is capped at six, so 15 pair evaluations $\times$ 2 BFS-style spread steps = roughly 50 graph traversals per turn, which runs in 0.5 ms on synthetic instances and 2 ms on the Köyceğiz graph. The top-k cap is the main hyperparameter; for much larger graphs ($n > 200$) a dynamic rule of `top_k = max(6, ceil(0.3 * front_size))` is the obvious next step.

4.9 Strategy 8 — Random (sanity baseline)

Random simply picks k vertices uniformly at random from the fire front F at each turn. It uses nothing about the graph topology, the state, or the budget beyond knowing which vertices are still white. Average burned is 70.1% on synthetic instances and 31.7% on the Köyceğiz case study, which is by far the worst of the eight. We include it for calibration: any informed strategy whose burned fraction is close to Random’s is, by definition, not extracting meaningful signal from the graph, and the corresponding scoring function would deserve to be cut from the ensemble. In practice every informed strategy in this report sits at least 25 points below Random on the synthetic average, so the calibration passes.

4.10 Strategy portfolio at a glance

Philosophy	Strategy	Question it answers
Local greedy	Max Degree	Which fire-front vertex has the most connections?
Local greedy	Max White Neighbours	Which fire-front vertex protects the most white area?
Structural	Min Cut Edge Front	Where is the bottleneck (edge-based)?
Structural	Min Damage Cut	Which cut saves the most vertices per unit cost?
Structural	Betweenness Front	Which vertex carries the most shortest-path traffic?
Hybrid	Hybrid Density Aware	Greedy or cut, depending on front density?
Lookahead	One-Step Lookahead	Which pair minimises burned two turns from now?